AI COURSE – II AIML/REC

CATEGORY 3 – PYTHON PROGRAMS ONLY

1. Exploration and Exploitation Trade-off

```
import random

# Example: choosing between three actions with unknown rewards.
actions = ["A", "B", "C"]
true_rewards = {"A": 3, "B": 6, "C": 9}
estimated_rewards = {action: 0 for action in actions}
counts = {action: 0 for action in actions}

epsilon = 0.2
total_steps = 30

for step in range(total_steps):
    # Exploration: choose a random action.
    # Exploitation: choose the action with the best estimated reward.
    if random.random() < epsilon:
        chosen = random.choice(actions)
        decision = "Explore"
    else:
        chosen = max(estimated_rewards, key=estimated_rewards.get)
        decision = "Exploit"

    # Simulate a reward with a little random variation.
    reward = true_rewards[chosen] + random.randint(-1, 1)

    counts[chosen] += 1
    n = counts[chosen]

    # Update the estimated average reward.
    estimated_rewards[chosen] += (
        reward - estimated_rewards[chosen]
    ) / n

    print("Step:", step + 1,
          "Mode:", decision,
          "Action:", chosen,
          "Reward:", reward)

print("\nEstimated rewards:", estimated_rewards)
print("Action counts:", counts)
print("Best estimated action:",
      max(estimated_rewards, key=estimated_rewards.get))
```

2. Hill Climbing Search – Eight Queens Problem

```
import random

N = 8

def heuristic(board):
    conflicts = 0
    for i in range(N):
        for j in range(i + 1, N):
            if board[i] == board[j]:
                conflicts += 1
            if abs(board[i] - board[j]) == abs(i - j):
                conflicts += 1
    return conflicts


def hill_climbing():
    board = [random.randint(0, N - 1) for _ in range(N)]

    while True:
        current_h = heuristic(board)
        best_board = board[:]
        best_h = current_h

        for col in range(N):
            for row in range(N):
                if row == board[col]:
                    continue

                neighbour = board[:]
                neighbour[col] = row
                h = heuristic(neighbour)
```

```
                if h < best_h:
                    best_h = h
                    best_board = neighbour

        if best_h >= current_h:
            return board, current_h

        board = best_board

solution, conflicts = hill_climbing()

print("Board:", solution)
print("Number of conflicts:", conflicts)

if conflicts == 0:
    print("Solution found")
else:
    print("Local optimum reached; restart may be needed")

for row in range(N):
    print(" ".join("Q" if solution[col] == row else "."
                   for col in range(N)))
```

3. Simulated Annealing Search

```
import math
import random

def objective(x):
    return -(x - 5) ** 2 + 25


def simulated_annealing():
    current = random.uniform(-10, 10)
    current_value = objective(current)

    best = current
    best_value = current_value

    temperature = 100.0
    cooling_rate = 0.95
    minimum_temperature = 0.001

    while temperature > minimum_temperature:
        candidate = current + random.uniform(-1, 1)
        candidate = max(-10, min(10, candidate))
        candidate_value = objective(candidate)

        difference = candidate_value - current_value

        if difference > 0 or random.random() < math.exp(
            difference / temperature
        ):
            current = candidate
            current_value = candidate_value

        if current_value > best_value:
            best = current
            best_value = current_value

        temperature *= cooling_rate

    return best, best_value


best_x, best_value = simulated_annealing()

print("Best x:", round(best_x, 4))
print("Maximum value:", round(best_value, 4))
```

4. Local Beam Search

```
import random

# Find a state with a low number of conflicts in the
# Eight Queens problem using Local Beam Search.

N = 8
K = 4

def heuristic(board):
    conflicts = 0
    for i in range(N):
        for j in range(i + 1, N):
            if board[i] == board[j]:
                conflicts += 1
            if abs(board[i] - board[j]) == abs(i - j):
                conflicts += 1
    return conflicts


def neighbours(board):
    result = []
    for col in range(N):
        for row in range(N):
            if row != board[col]:
                new_board = board[:]
                new_board[col] = row
                result.append(new_board)
    return result


def local_beam_search():
    beam = [
        [random.randint(0, N - 1) for _ in range(N)]
        for _ in range(K)
    ]
```

```
    for step in range(1000):
        beam.sort(key=heuristic)

        if heuristic(beam[0]) == 0:
            return beam[0]

        all_neighbours = []
        for board in beam:
            all_neighbours.extend(neighbours(board))

        all_neighbours.sort(key=heuristic)
        beam = all_neighbours[:K]

    return min(beam, key=heuristic)


solution = local_beam_search()

print("Best board:", solution)
print("Number of conflicts:", heuristic(solution))

if heuristic(solution) == 0:
    print("Solution found")
else:
    print("Best state after maximum iterations")

for row in range(N):
    print(" ".join("Q" if solution[col] == row else "."
                   for col in range(N)))
```